Simulator 3: Multi-Agent Research System

Alignment: Domains 1, 2 and 5 · Runs on the Claude Agent SDK through your Claude login, against mock backends

Objective

Run a real coordinator on the Claude Agent SDK that splits "impact of AI on creative industries" across four web-search subagents and one document-analysis subagent in a single response, survives a timed-out search by reading its structured error, hands the merged findings to a synthesis subagent that owns one scoped tool, and ends with a report in which every statistic keeps its source and date and two conflicting adoption rates stay side by side.

Scenario (official exam scenario 3)

"You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports."

Primary domains: Agentic Architecture & Orchestration, Tool Design & MCP Integration, Context Management & Reliability.

Prerequisites

- `uv` and a Claude login (`claude auth login`). Run `uv sync` once in the `labs` folder. No API key.
- Open this simulator in your browser with `uv run serve.py` from `labs`, or run the commands below from this simulator's folder.

What is in the folder

File	Role
`research_system.py`	In-process MCP server (`search_web`, `load_document`, `verify_fact`), four `AgentDefinition`s, the coordinator prompt, stream capture
`fixtures/sources.json`	The sandbox web (music, writing, film; visual arts wired to time out), two PDFs with conflicting SME adoption rates (41% vs 55%), the fact bank
`cases/UC-*.json`	Use-case files behind this simulator's labs: situation, question, correct option, task, rubric
`rules_sim.py`	Drill for Q7: materialises `repo-rules/`, tries four placements of the same testing guidance, runs Claude twice per placement through the Agent SDK
`verify.py`	Twelve acceptance checks (thirteen with `--narrow`) read from `evidence/`, plus a summary of any use-case results
`evidence/`	`run.json`, `report.md`, optional `run-narrow.json`, `verification.txt`

© 2026 Tertiary Infotech Academy · UEN 201200696W

Guided tour

1. Read the three tool descriptions. Each says what the tool is for, what it returns, and what it is not for; `search_web` documents its error shape so an agent can decide between retry, rephrase and gap.

```
sed -n '/@tool(/,/^)/p' research_system.py | head -60
```

2. Read the four `AgentDefinition`s. web-search holds only `search_web`, document-analysis only `load_document`, synthesis only `verify_fact`, report nothing. Every prompt demands JSON out and forbids inventing numbers.

```
grep -n -A4 'AgentDefinition(' research_system.py
```

3. Run the coordinator. Watch the coordinator lines count up to five Agent calls on the same response id (the CLI streams one line per tool call, so the count climbs 1, 2, 3, 4, 5 for one response). A few lines later a structured timeout error appears for the visual-arts search.

```
uv run research_system.py
```

4. Prove the parallel launch from the evidence. `agent_calls_in_this_response` on the first turn is 5; the `packets` list shows the prompt size handed to each subagent, because a subagent starts with an empty context.

```
python3 -c "import json; r=json.load(open('evidence/run.json'));
    print([t['agent_calls_in_this_response'] for t in r['assistant_turns']]);
    p=r['assistant_turns'][0]['packets'][0]; print(p['subagent_type'], p['prompt_chars'],
    p['prompt_head'])"
```

5. Read the structured error the web agent received: failure type, attempted query, empty partial results, three alternatives. The run did not abort; the coordinator carried the gap into synthesis.

```
python3 -c "import json; r=json.load(open('evidence/run.json'));
    print(r['structured_errors'][0])"
```

6. Open the report. Find the table with Source and Published columns, the Contested findings section holding 41% (WIPO) and 55% (UNESCO) side by side, and the Coverage gaps section naming visual arts.

```
cat evidence/report.md
```

7. Check which tools actually ran. The synthesis agent's two `verify_fact` calls are at the end; no agent called a tool outside its definition.

```
python3 -c "import json; [print(c['tool'], c.get('query') or c.get('doc_id') or
    c.get('claim')) for c in json.load(open('evidence/run.json'))['tool_calls']]"
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

8. Reproduce the exam guide's failure. The narrow coordinator splits the topic into digital art, graphic design and photography only. Every subagent succeeds, yet music, writing and film never get searched.

```
uv run research_system.py --narrow
grep -o '"query": "[^"]*"' evidence/run-narrow.json | sort -u
```

9. Run the verifier.

```
uv run verify.py
```

10. Stretch: in `fixtures/sources.json` replace the visual-arts `"TIMEOUT"` with `[]` and rerun. The search now returns a valid empty result and the report should say so, not report a failure.

Labs that run on this simulator

Open this simulator in your browser: from the `labs` folder run `uv run serve.py`, then pick Simulator 3. In Claude Code, type `/lab` and the simulator number. A simulator is the system. A lab is one exam situation you run on it. Run any of these from the `labs` folder with `uv run run_lab.py <lab>`; each has its own PDF under its topic folder. An asterisk marks a lab run in class.

Lab	Task	Source	What it proves	Command
3*	1.2	Guide Q7	Every subagent succeeds, and the report still misses half the topic	`uv run research_system.py --narrow`
7	1.3	Practice Q10	Polished report, weak citations: route information, not just tasks	`uv run research_system.py --case UC-10`
8	1.3	Practice Q35	Later subagents re-run searches the first ones already finished	`uv run research_system.py --case UC-35`
22	2.3	Guide Q9	Give synthesis one narrow tool, not the whole web	`uv run research_system.py`
30*	3.3	Practice Q7	Testing rules for test files spread across every folder	`uv run rules_sim.py D`
35	3.5	Practice Q27	Two linked defects go in one detailed request	`uv run research_system.py --case UC-27`
39	4.1	Practice Q20	Switch off the one noisy category, keep the accurate ones	`uv run research_system.py --case UC-20`
41	4.3	Practice Q28	Vague passages forced into a category they don't fit	`uv run research_system.py --case UC-28`
48*	4.6	Practice Q6	The author reviewing its own draft misses its own errors	`uv run research_system.py --case UC-06`
49	4.6	Practice Q19	Source-level passes first, integration second	`uv run research_system.py --case UC-19`

© 2026 Tertiary Infotech Academy · UEN 201200696W

Lab	Task	Source	What it proves	Command
53	5.1	Practice Q21	Taxonomy drift: write it to a state file	`uv run research_system.py --case UC-21`
55*	5.3	Guide Q8	The search timed out: tell the coordinator everything	`uv run research_system.py`
61*	5.6	Practice Q24	A vendor benchmark isn't a peer-reviewed study	`uv run research_system.py --case UC-24`

where does testing guidance belong? (Q7)

The research system's TypeScript repo keeps citation-verification, report-rendering and orchestration tests beside their implementation files in three top-level folders. The team wants one piece of testing guidance to apply only when Claude Code works on a test file. `rules_sim.py` builds that repo under `repo-rules/` and tries the four answers from the practice test. The guidance carries a token, `TESTING-GUIDANCE-LOADED`, and asks Claude to name the file it came from, so each reply tells you whether the guidance was in context.

Variant	Guidance placed in	Copies
A	`.claude/rules/testing.md` with no frontmatter	1
B	root `CLAUDE.md`	1
C	a `CLAUDE.md` in every folder	3
D	`.claude/rules/testing.md` with `paths: ["**/*.test.ts"]`	1

Each variant gets two runs limited to Read and Edit: add an assertion to `orchestration/orchestrator.test.ts`, then change a header line in `report-rendering/renderer.ts`.

11. Run the simulator (eight short calls, about USD 3 on Opus).

```
uv run rules_sim.py
```

12. Read the matrix it prints and `evidence/rules_sim.json`. Expected: A and B load on both edits, C loads on both and needs three copies, D loads only on the test-file edit with one copy. The matrix observed while building this simulator:

```
variant | guidance placed in                       | copies | test edit | impl edit
   A    | .claude/rules/testing.md, no frontmatter |   1    |   yes     |   yes
   B    | root CLAUDE.md                           |   1    |   yes     |   yes
   C    | CLAUDE.md in every folder                |   3    |   yes     |   yes
   D    | .claude/rules/testing.md, paths: **/*.test.ts |   1 |   yes     |   no
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

13. Open the D rule file and the reply that named it. A rule file without frontmatter (A) is just always-on memory; the file name scopes nothing. Root `CLAUDE.md` (B) loads for every session. Per-folder `CLAUDE.md` (C) loads for any file in the folder, implementation files included, and must be maintained three times.

```
sed -n '1,6p' repo-rules/.claude/rules/testing.md
python3 -c "import json;
    d=json.load(open('evidence/rules_sim.json'))['variants']['D']['runs'];
    print(d['test']['result'][:300]); print('---'); print(d['impl']['result'][:300])"
```

14. The verifier adds three checks for this matrix, and `--case UC-07` evaluates the same rubric from `evidence/rules_sim.json`.

Expected evidence

- `evidence/run.json`: coordinator tool list, agent definitions, per-turn Agent call counts, packet sizes, structured errors, every MCP call, cost and session id.
- `evidence/cases/UC-<n>.json`: one record per use case with the filled prompt, mode, cost and rubric results.
- `evidence/rules_sim.json`: the four-variant guidance matrix with each reply and its cost.
- `evidence/report.md`: the cited report with established, contested and coverage-gap sections.
- `evidence/run-narrow.json` and `evidence/report-narrow.md` after step 8.
- `evidence/verification.txt`: PASS lines from step 9.

Acceptance checks

- `Agent` is in the coordinator's allowed tools; each subagent holds at most one tool.
- At least three Agent calls appear in one coordinator response, and the web searches cover music, writing and film.
- The timeout arrives as a structured error with failure type, attempted query, partial results and alternatives, and the run still succeeds with the gap named.
- 41% and 55% both reach the report with their sources and dates; `verify_fact` was called by synthesis.
- With `--narrow`, no music, writing or film query is ever issued.

Exam link

Task statements 3.1 and 3.3 (`.claude/rules/` with `paths:` frontmatter versus CLAUDE.md placement, drill Q7), 1.2 (hub-and-spoke, decomposition breadth), 1.3 (Agent tool, explicit context packets, parallel spawning in one response), 2.1 (tool descriptions that state use and non-use), 2.3 (one scoped tool per role), 5.3 (structured error context, partial results, coverage annotations), 5.6 (claim-source-date mappings, conflicts kept side by side).

© 2026 Tertiary Infotech Academy · UEN 201200696W